

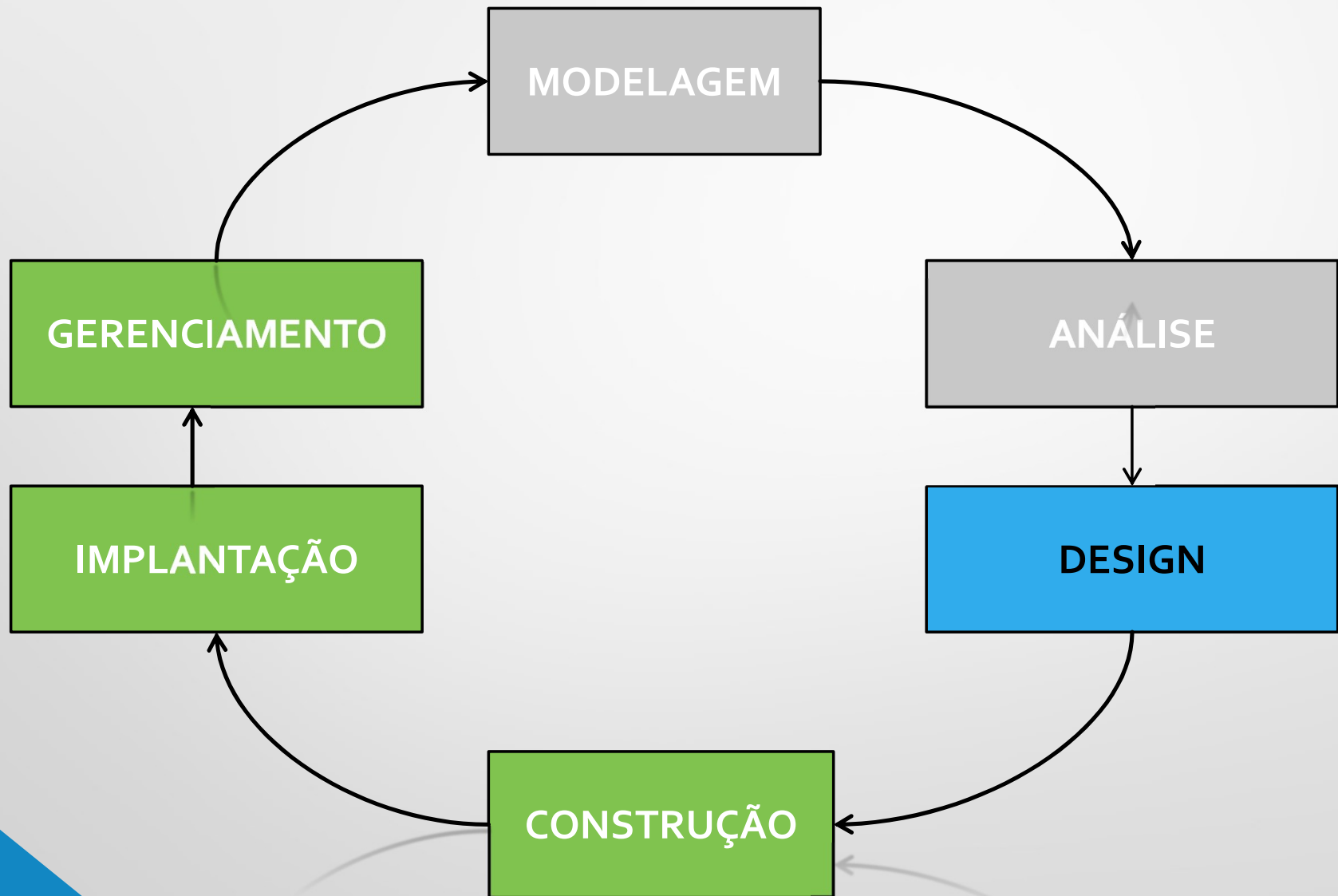


Tópico 3.3:

Engenharia de Software Orientada a Serviços

Fase de Design

Fases do Ciclo de Vida



Da Análise para o Design



Na Análise eu
descubro **O QUE**
o sistema faz.



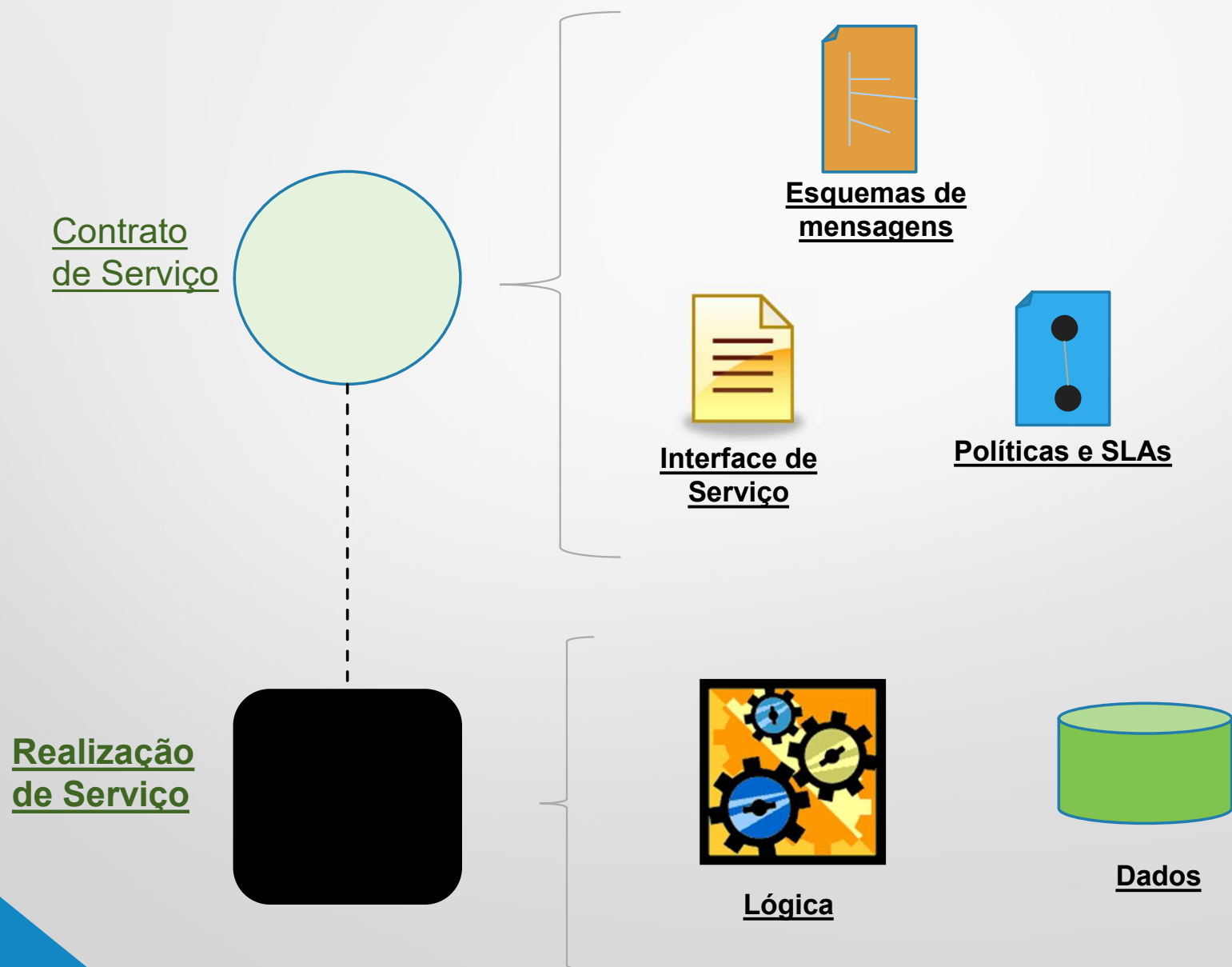
No Design eu defino
COMO os serviços
fazem isso
(especificação técnica)

Atividades da Fase de Design

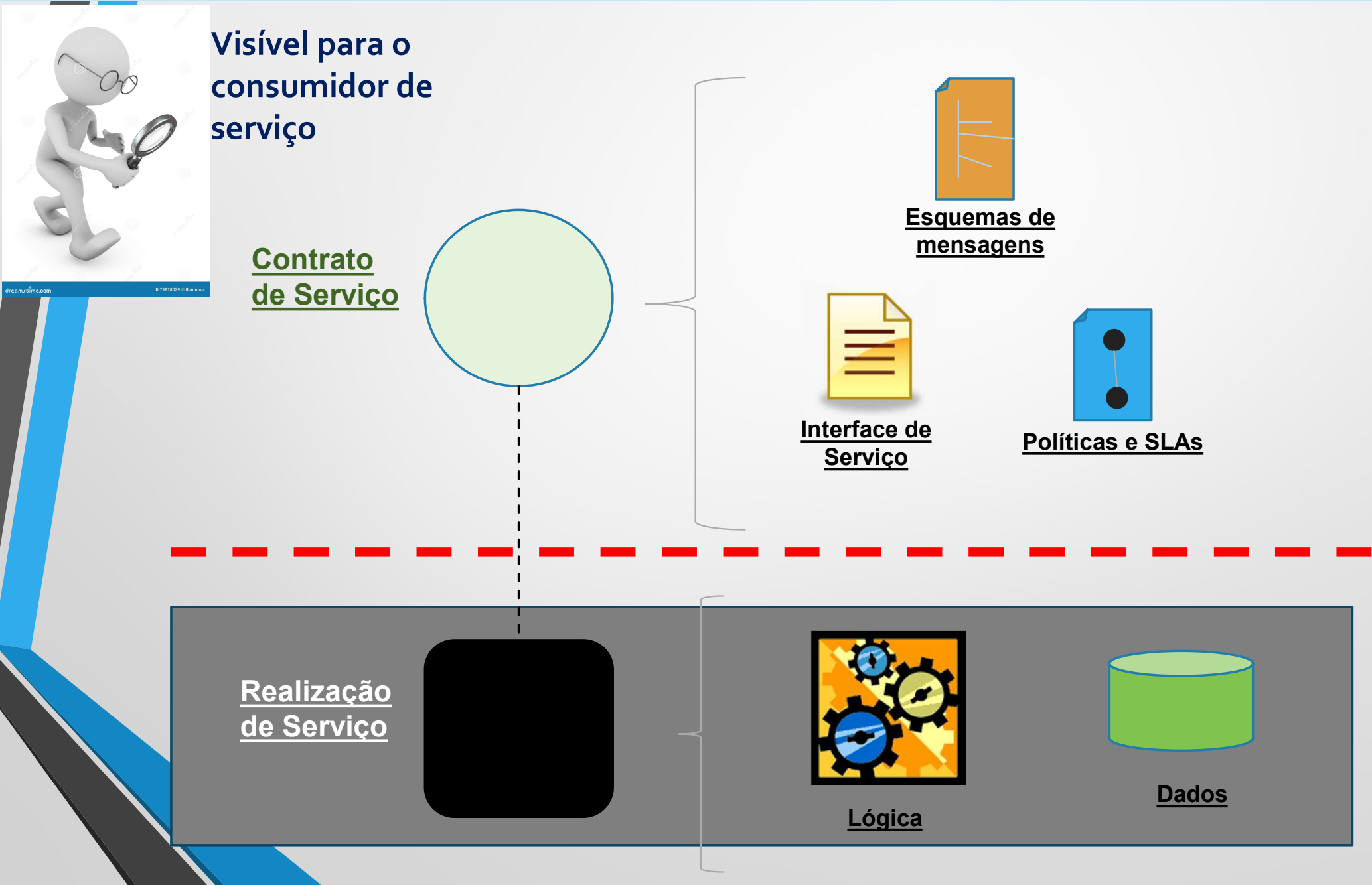
Transformar serviços candidatos em serviços definidos

1. Projetar esquemas de mensagens
2. Definir interface do serviço
3. Especificar políticas e SLA
4. Consolidar o contrato de serviço
5. Definir realização do serviço

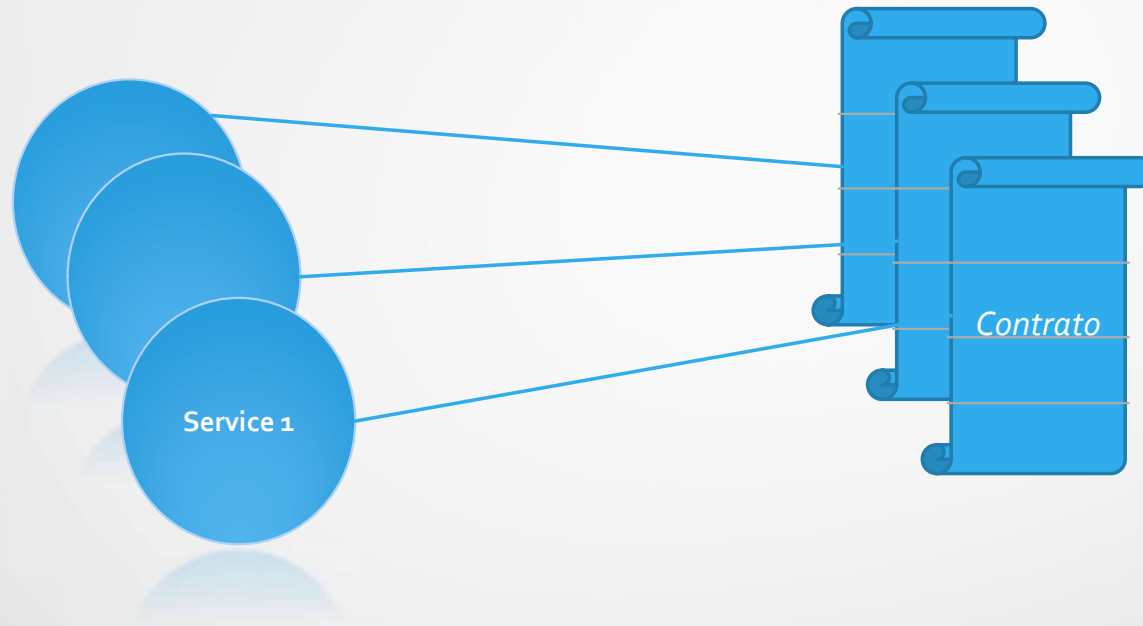
Anatomia de um Serviço



Anatomia de um Serviço



Contrato de Serviço



O contrato define o que o serviço expõe.

Contrato de Serviço

1. Projetar Esquemas de Mensagens:

São os “formatos dos dados” que os serviços enviam e recebem.

Ou seja:

- O que entra no serviço
- O que sai do serviço
- Como esses dados estão organizados

Contrato de Serviço

Esquemas de Mensagem: Cliente Service



<<Mensagem>> DadosCliente	
- nome	string
- cpf	string
- email	string
- telefone	string
- endereco	string

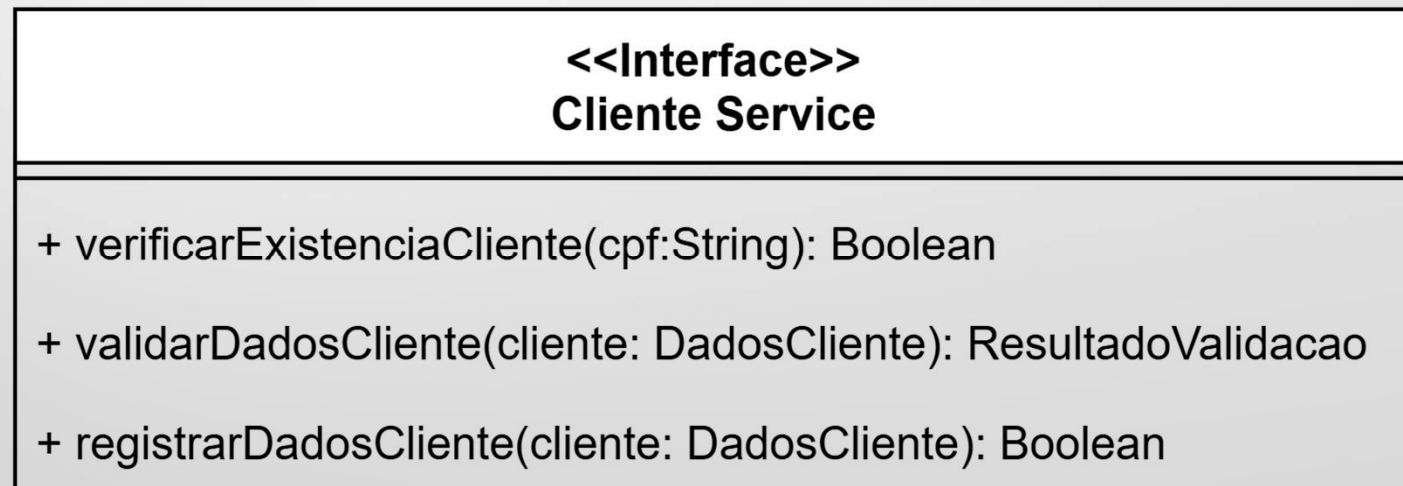
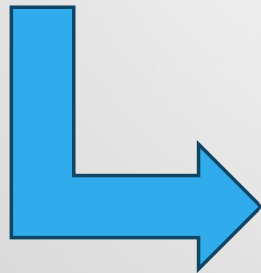
Contrato de Serviço

2. Projetar Interfaces de Serviço:

- Define as operações disponíveis no serviço.
- Especifica:
 - a. Nome da operação
 - b. Parâmetros de entrada
 - c. Dados de saída
- Utiliza esquemas de mensagens
- É visível para o consumidor

Contrato de Serviço

Interface do serviço “Paciente”



Contrato de Serviço

3. Especificar Políticas de Serviço:

- São regras que definem como o serviço deve ser acessado e utilizado.
- São as regras de uso do serviço.
 - ✓ Autenticação (protocolo OAuth 2.0, Token, API Key)
 - ✓ Autorização (quem pode acessar: Papéis RBAC)
 - ✓ Criptografia (HTTPS)
 - ✓ Limites de uso

Contrato de Serviço

Políticas

1. Cliente Service

Autenticação

- OAuth 2.0 com Token JWT
- Obrigatório para todas as operações

Autorização (RBAC)

- ATENDENTE → cadastrar, atualizar
- BACKOFF → consultar

Criptografia

- HTTPS obrigatório (TLS 1.2+)

Limite de uso

- 500 requisições por minuto por cliente

2. Consultar Crédito Service (SPC/Serasa)

Autenticação

- OAuth 2.0 + API Key do provedor

Autorização

- Apenas serviços internos autorizados
- NÃO acessível diretamente por usuários

criptografia

- HTTPS obrigatório
- Criptografia de dados sensíveis (CPF)

Limite de uso

- 100 requisições por minuto
- Controle rigoroso para evitar bloqueio externo

Contrato de Serviço

4. Definição de SLA

- O SLA (*Service Level Agreement*) é o acordo que define os níveis de qualidade e desempenho que o serviço deve garantir.
 - ✓ Disponibilidade
 - ✓ Tempo de resposta
 - ✓ Taxa de sucesso
 - ✓ Capacidade de processamento
 - ✓ Suporte Técnico

Contrato de Serviço

SLA (Service Level Agreement)

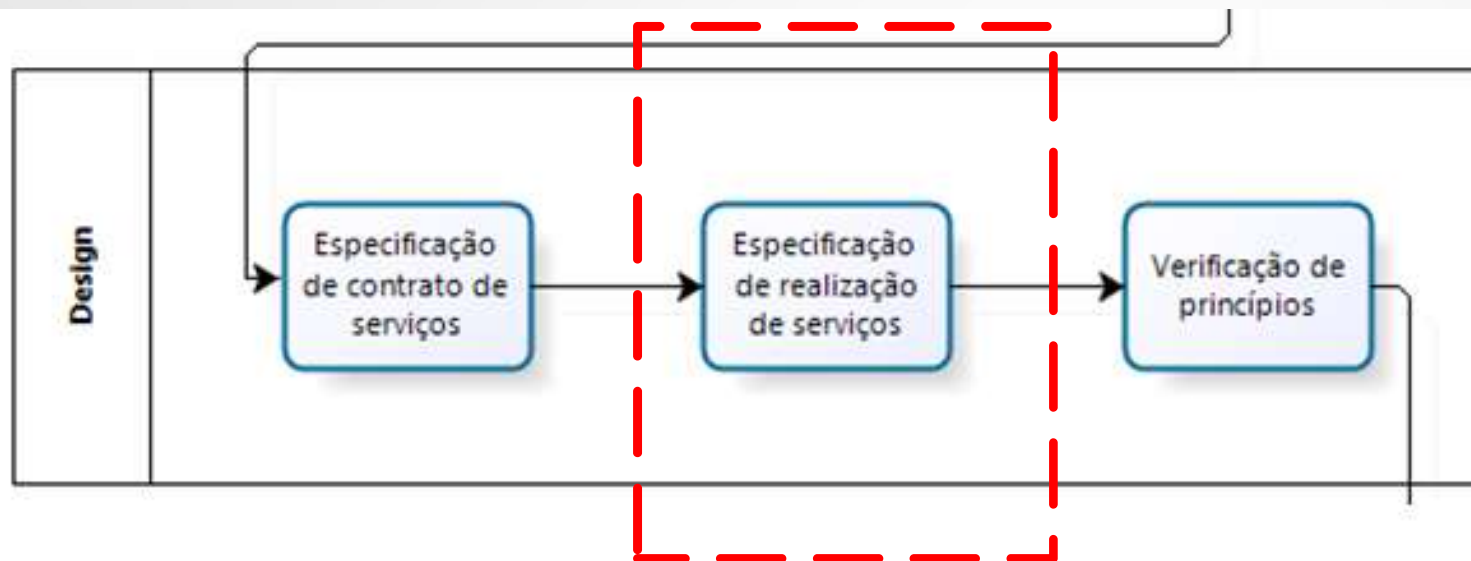
Cliente Service

- Disponibilidade: 98%
- Tempo de resposta: até 2s
- Taxa de sucesso: 98%
- Capacidade: 500 req/min
- Suporte: horário comercial
(8h–18h)

Consulta de Crédito Service

- Disponibilidade: 95%
- Tempo de resposta: até 5s
- Taxa de sucesso: 90%
- Capacidade: 100 req/min
- Suporte: depende do
provedor

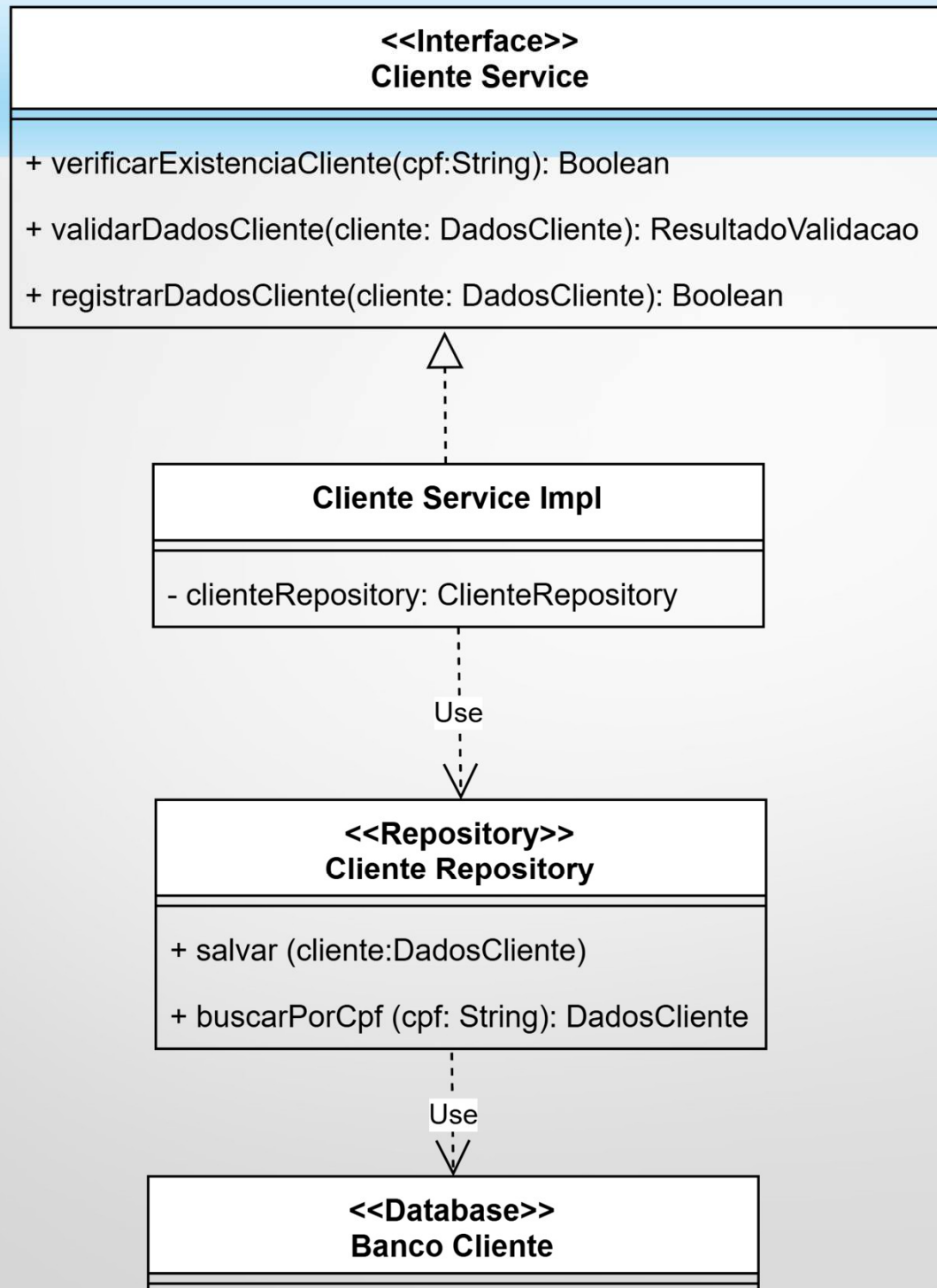
Design de um Serviço

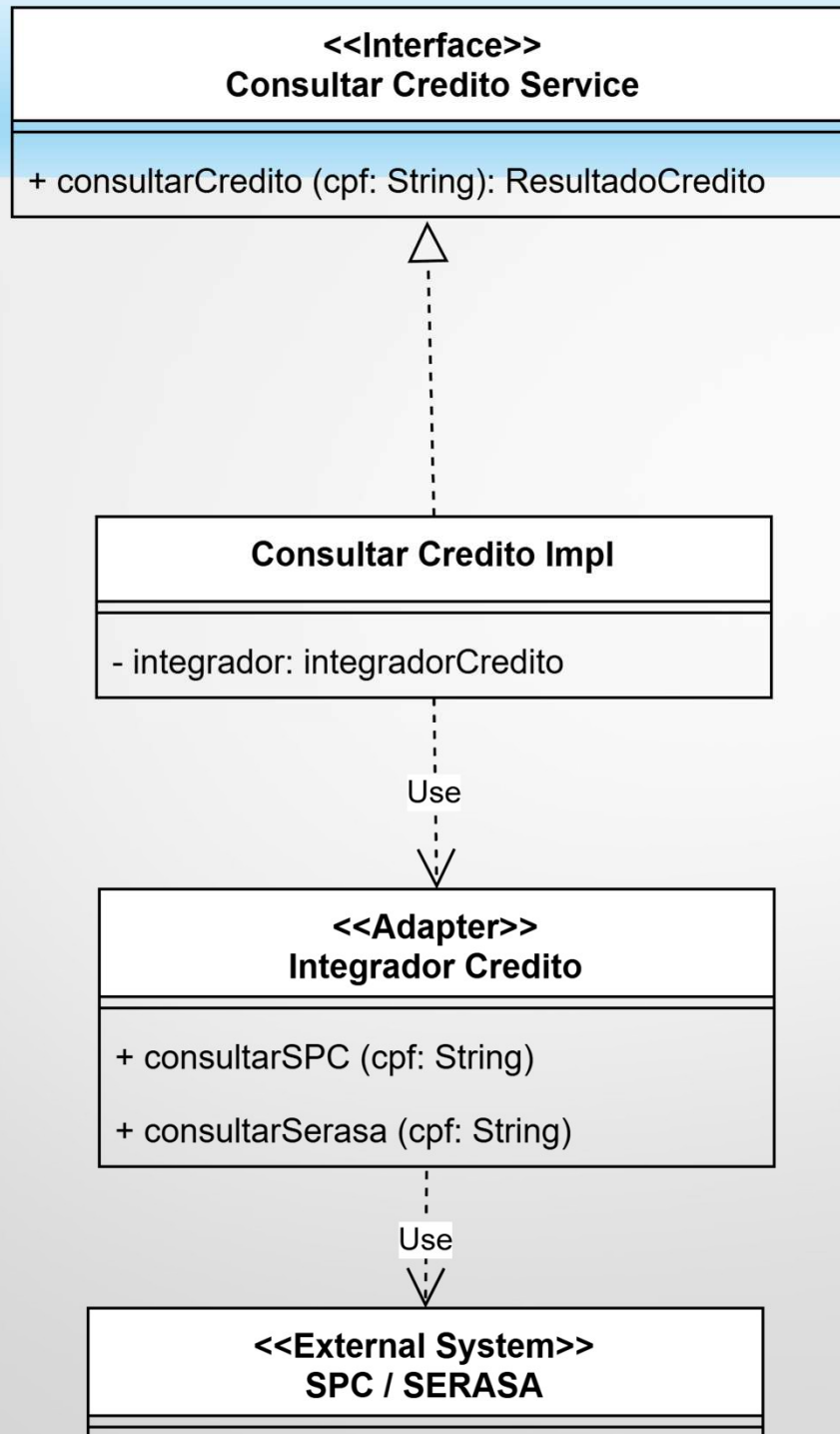


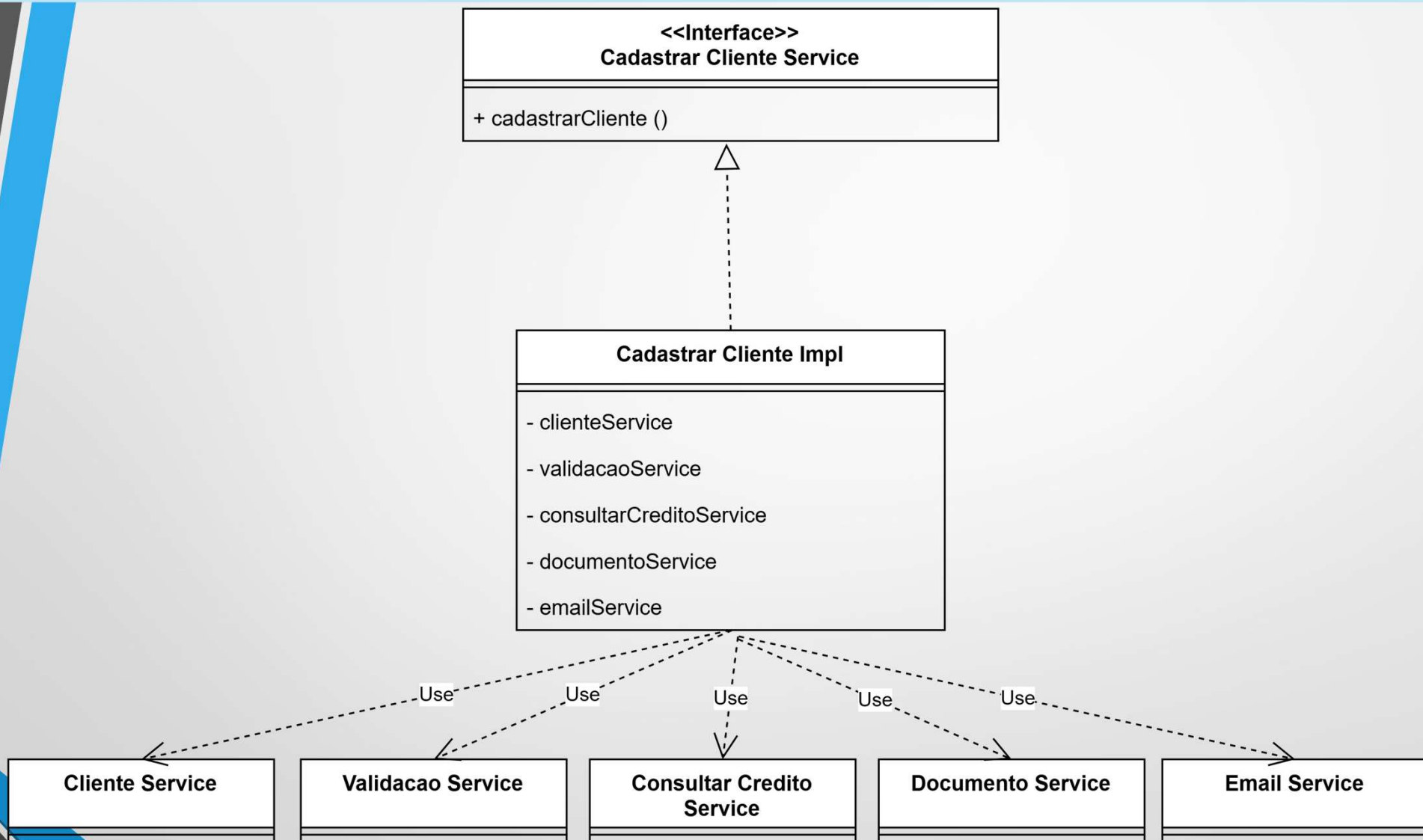
Design de um Serviço

Realização de Serviço:

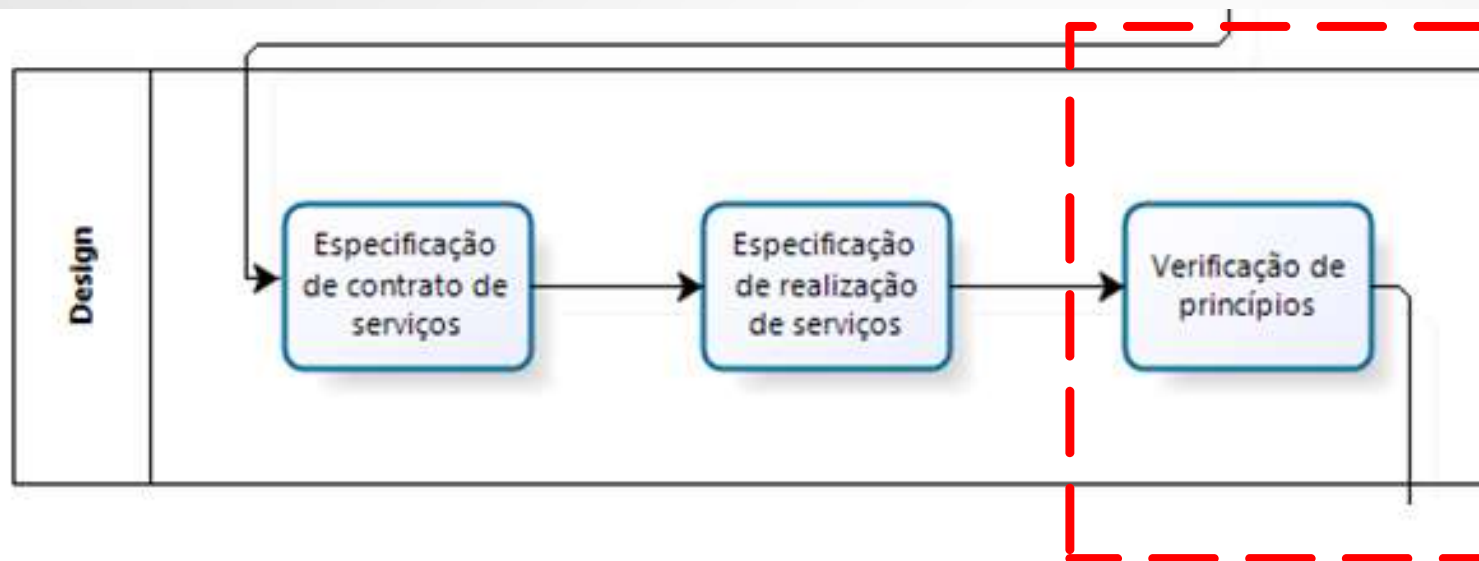
- Define como o serviço é implementado internamente.
- Inclui:
 - a. Lógica de negócio
 - b. Acesso a dados
 - c. Integrações
 - d. Orquestração
- Deve atender às políticas e SLA







Design de um Serviço



Design de um Serviço



Princípios da Orientação a Serviços

1. Contratos Padronizados

- ❑ Os contratos dos serviços (interfaces, formatos de mensagens, regras) devem ser bem definidos, claros e estáveis.
- ❑ Consumidores sabem exatamente quais dados enviar e receber sem precisar adivinhar como o serviço funciona internamente.

Princípios da Orientação a Serviços

2. Reusabilidade

- ☐ Criar serviços genéricos, não muito específicos
- ☐ Evitar dependências fortes com um único sistema
- ☐ Usar interfaces bem definidas
- ☐ Documentar bem o serviço

Princípios da Orientação a Serviços

3. Granularidade

- ❑ Granularidade, refere-se ao tamanho da tarefa que o serviço executa.
- ❑ Alta granularidade → serviço pequeno e específico (muito reusável)
 - validarCPF()
 - calcularFrete()
 - verificarEstoque()
- ❑ Baixa granularidade → serviço grande e mais completo (menos flexível para reutilização)
 - processarPedido()
(inclui validar cliente + calcular frete + verificar estoque)

Princípios da Orientação a Serviços

4. Autonomia

- ❑ Um serviço deve ser capaz de **executar suas funções de forma independente**, mantendo controle sobre sua própria lógica e recursos, com o mínimo de dependência dos detalhes internos de outros serviços.

Ex: um serviço de agendamento que solicita a autorização de exames a um serviço de plano de saúde por meio de uma interface, sem depender da lógica interna desse sistema, utilizando apenas o resultado retornado.

Princípios da Orientação a Serviços

5. Abstração

- ❑ Os detalhes internos de implementação do serviço devem ser ocultos; o consumidor deve interagir apenas com a interface.
- ❑ Ex: Usuário chama um serviço para gerar relatório, sem saber como os dados são processados internamente.

Princípios da Orientação a Serviços

6. Baixo Acoplamento

- ☐ Significa que os serviços devem depender o mínimo possível uns dos outros, evitando que mudanças em um serviço afetem outros consumidores.
- ☐ Consumidores só dependem do contrato (interface pública).
- ☐ A implementação interna pode mudar sem quebrar os consumidores.

Princípios da Orientação a Serviços

7. Visibilidade

- ❑ Um serviço deve ser facilmente descoberto e ter suas funcionalidades, interfaces e condições de uso claramente descritas, para que outros serviços ou aplicações possam encontrá-lo e utilizá-lo corretamente.
 - Possui documentação clara da API.
 - Registrado em um repositório de serviços (service registry)

Princípios da Orientação a Serviços

8. Capacidade de Composição

- ❑ Serviços podem ser combinados para formar serviços mais complexos ou processos de negócios maiores.
- ❑ **Ex:** Um “processo de agendar consulta” pode usar serviços de autenticação, paciente, médico, agenda e notificação.

Princípios da Orientação a Serviços

9. Independência de Estado (Stateless)

- ❑ Um serviço não deve depender do estado interno de outro serviço para funcionar.
- ❑ Cada serviço deve manter seu próprio estado ou operar de forma **stateless**, garantindo que interações entre serviços sejam previsíveis e confiáveis.

Princípios da Orientação a Serviços

10. Interoperabilidade

- ☐ Contratos de serviços devem ser desenhados para serem interoperáveis, usando padrões abertos.
- ☐ Serviços devem ser capazes de funcionar em diferentes plataformas, linguagens e ambientes.
- ☐ **Ex:** Um serviço SOAP em Java pode ser consumido por um cliente .NET.